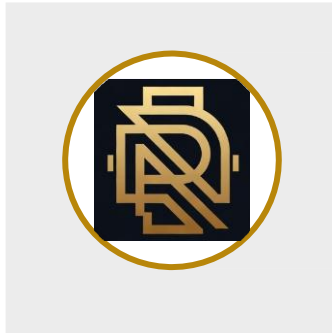




ROYALRENT

Plataforma de Alquiler de Vehículos y Servicios Premium

Proyecto de Fin de Ciclo

Desarrollo de Aplicaciones Multiplataforma (DAM)

Autores:

Gonzalo Velasco Paniagua

Miguel José Martínez Majan

Tutor:

Félix De Pablo Lobo

Grupo: 05 · **Curso:** 2024–2026

Universidad Europea de Madrid

1. Introducción

RoyalRent es una aplicación web desarrollada como Proyecto de Fin de Ciclo del Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM). Se trata de una plataforma pensada para el alquiler de vehículos de lujo —coches, yates y helicópteros— que permite a los usuarios explorar un catálogo completo, gestionar un carrito de compra, registrarse, iniciar sesión y realizar reservas de forma sencilla y ágil.

La aplicación se compone de dos partes bien diferenciadas. El frontend es una aplicación SPA (Single Page Application) construida con React, que ofrece una interfaz moderna, adaptable a cualquier dispositivo y con animaciones cuidadas. El backend, por su parte, es una API REST desarrollada con Node.js y Express, que se encarga de gestionar usuarios, productos, pedidos y la autenticación mediante tokens JWT.

El catálogo incluye veinte coches de marcas como Ferrari, Lamborghini o Rolls-Royce; doce yates de astilleros como Azimut, Sunseeker o Ferretti; y ocho helicópteros de fabricantes como Bell, Airbus o AgustaWestland. Todos ellos cuentan con descripciones detalladas, precio por día e imágenes representativas. El sistema cuenta además con un panel de administración desde el que los gestores pueden supervisar usuarios, productos, pedidos y mensajes de contacto.

1.1. Motivación

La idea de RoyalRent nace de la necesidad de digitalizar el proceso de alquiler de vehículos de alta gama, un sector que tradicionalmente ha dependido de procesos manuales y poco eficientes. Una plataforma digital permite agilizar las reservas, mejorar la experiencia del cliente y ofrecer herramientas de gestión a los administradores del negocio.

Además, este proyecto permite poner en práctica y consolidar los conocimientos adquiridos durante el ciclo formativo, combinando programación, bases de datos, desarrollo web y diseño de interfaces en un único proyecto completo.

1.2. Objetivos

Los objetivos que se persiguen con este proyecto son los siguientes:

- Diseñar y desarrollar una aplicación web completa y funcional, desde la base de datos hasta la interfaz de usuario.

- Implementar un sistema de autenticación seguro basado en tokens JWT con contraseñas protegidas mediante bcrypt.
- Crear una API REST para la gestión de usuarios, productos y pedidos.
- Diseñar una base de datos relacional que refleje correctamente las entidades del sistema y sus relaciones.
- Desarrollar una interfaz de usuario clara, moderna y adaptable a cualquier dispositivo.
- Aplicar buenas prácticas de desarrollo de software, incluyendo control de versiones y documentación.

Enfoque del proyecto. RoyalRent se enmarca dentro de un proyecto de desarrollo aplicado: la creación de una aplicación funcional relacionada con el campo de estudio del alumno. El proyecto se apoya en situaciones reales y se organiza siguiendo un plan de trabajo por fases.

2. Palabras Clave

A continuación se explican los acrónimos y términos técnicos que aparecen en este documento:

Término	Definición
API	Application Programming Interface. Conjunto de reglas que permite la comunicación entre distintas aplicaciones.
CRUD	Create, Read, Update, Delete. Operaciones básicas para gestionar datos.
CSS	Cascading Style Sheets. Lenguaje que define la presentación visual de los documentos HTML.
DAM	Desarrollo de Aplicaciones Multiplataforma. Ciclo formativo de grado superior.
DOM	Document Object Model. Representación en árbol de un documento HTML que permite manipularlo con JavaScript.
HTTP	HyperText Transfer Protocol. Protocolo de comunicación utilizado en la web.
IDE	Integrated Development Environment. Entorno de desarrollo que agrupa herramientas de programación.
JSON	JavaScript Object Notation. Formato ligero para intercambiar datos.
JWT	JSON Web Token. Estándar para crear tokens de autenticación con información codificada.
PFC	Proyecto de Fin de Ciclo. Módulo profesional del segundo curso.
REST	Representational State Transfer. Estilo de arquitectura para diseñar APIs web.
SPA	Single Page Application. Aplicación web que funciona en una sola página y actualiza el contenido dinámicamente.
SQL	Structured Query Language. Lenguaje estándar para trabajar con bases de datos relacionales.

3. Índice General

1. Introducción

- 1.1. Motivación
- 1.2. Objetivos

2. Palabras Clave

3. Índice General

4. Módulos Formativos Aplicados

5. Herramientas y Lenguajes Utilizados

- 5.1. Lenguajes de programación
- 5.2. Frameworks y bibliotecas
- 5.3. Herramientas de desarrollo

6. Componentes del Equipo

7. Fases del Proyecto

- 7.1. Modelo de datos
- 7.2. Diagrama de clases
- 7.3. Casos de uso
- 7.4. Diseño de las interfaces
- 7.5. Planificación del desarrollo
- 7.6. Funcionalidad del proyecto

8. Conclusiones y Mejoras

9. Bibliografía

10. Anexos

4. Módulos Formativos Aplicados

Para desarrollar RoyalRent se han utilizado conocimientos de los siguientes módulos del ciclo de Desarrollo de Aplicaciones Multiplataforma:

Módulo	Cómo se ha aplicado
Programación	El backend está desarrollado con Node.js y Express. Se ha implementado toda la lógica de negocio, los endpoints de la API, el manejo de errores y la programación asíncrona.
Bases de Datos	Se ha diseñado un modelo relacional en MySQL con seis tablas, sus relaciones y claves foráneas. Las operaciones críticas usan transacciones ACID.
Desarrollo Web en Entorno Cliente	La interfaz está construida con React, usando componentes reutilizables, Context API para el estado global, React Router para la navegación y Framer Motion para las animaciones.
Desarrollo Web en Entorno Servidor	El servidor Express incluye middleware de autenticación, un pool de conexiones a MySQL y la gestión completa de peticiones y respuestas HTTP.
Sistemas de Gestión Empresarial	Se ha modelado un negocio de alquiler premium con usuarios, roles, pedidos y un panel de administración con indicadores.
Programación de Servicios y Procesos	El servidor gestiona peticiones concurrentes y procesos asíncronos, y la comunicación con el frontend se realiza mediante servicios REST.
Acceso a Datos	Se ha implementado un pool de conexiones reutilizables y transacciones para garantizar la integridad de los datos en operaciones críticas.
Desarrollo de Interfaces	El diseño es responsivo, con animaciones cuidadas, una paleta de colores corporativa y una experiencia de usuario pensada para dispositivos móviles y de escritorio.

5. Herramientas y Lenguajes Utilizados

5.1. Lenguajes de programación

Lenguaje	Qué hace en el proyecto
JavaScript (ES6+)	Es el lenguaje principal, usado tanto en el frontend como en el backend. Se han empleado características modernas como arrow functions, async/await y módulos.
HTML5	Define la estructura de las páginas y los componentes React mediante JSX.
CSS3	Da estilo a la aplicación con Flexbox, Grid, animaciones y diseño adaptativo.
SQL	Se usa para crear, consultar, actualizar y eliminar datos en MySQL.

5.2. Frameworks y bibliotecas

Frontend

Tecnología	Versión	Función
React	19.2.5	Biblioteca principal para construir la interfaz con componentes reutilizables.
React Router	7.15.0	Gestiona la navegación entre páginas sin recargar el navegador.
Axios	1.16.0	Cliente HTTP para comunicarse con el backend.
Framer Motion	12.40.0	Añade animaciones y transiciones a los componentes.
Lucide React	1.14.0	Proporciona iconos vectoriales para la interfaz.
Vite	8.0.10	Entorno de desarrollo y empaquetador rápido.

Backend

Tecnología	Visión	Función
Node.js	23.x	Entorno de ejecución para JavaScript en el servidor.
Express	5.2.1	Framework para crear la API REST.
MySQL2	3.22.3	Driver para conectar con MySQL usando promesas y pool de conexiones.
bcrypt	6.0.0	Protege las contraseñas mediante hash con sal.
jsonwebtoken	9.0.3	Genera y verifica los tokens de autenticación JWT.
cors	2.8.6	Permite peticiones desde el frontend al backend (cross-origin).
dotenv	17.4.2	Carga variables de configuración desde archivos .env.

5.3. Herramientas de desarrollo

Herramienta	Para qué se ha usado
Visual Studio Code	Editor de código con extensiones para ESLint, IntelliSense y depuración.
ESLint	Revisa el código para mantener calidad y buenas prácticas.
Git	Sistema de control de versiones.
GitHub	Plataforma para alojar el repositorio y trabajar en equipo.
XAMPP	Servidor local con MySQL para desarrollar y probar.

6. Componentes del Equipo

El proyecto ha sido realizado por dos alumnos. Estas han sido sus aportaciones:

Miembro	Rol	Qué ha hecho
Gonzalo Velasco Paniagua	Desarrollador Full Stack	Se ha encargado del backend: la API REST con Express, el diseño de la base de datos, la autenticación JWT, los endpoints seguros, la configuración del servidor y la lógica de negocio.
Miguel José Martínez Maján	Desarrollador Full Stack	Ha desarrollado el frontend: los componentes de React, el diseño de las interfaces, las animaciones, la maquetación responsiva y la integración con la API.

Ambos han colaborado en las tareas de análisis, diseño, pruebas e integración del proyecto.

7. Fases del Proyecto

7.1. Modelo de datos

La base de datos de RoyalRent está formada por seis tablas que recogen las entidades principales del sistema. A continuación se describe cada una.

Tabla: users (usuarios)

Guarda los datos de los usuarios registrados.

Campo	Tipo	Restricciones	Descripción
id	INT	Clave primaria, autoincremental	Identificador único del usuario
name	VARCHAR(100)	Obligatorio	Nombre completo
email	VARCHAR(100)	Único, obligatorio	Correo electrónico
password	VARCHAR(255)	Obligatorio	Contraseña cifrada con bcrypt
role	ENUM('user','admin')	Valor por defecto: 'user'	Rol del usuario
created_at	TIMESTAMP	Valor por defecto: fecha actual	Fecha de registro

Tabla: items (productos)

Catálogo de vehículos disponibles para alquilar.

Campo	Tipo	Restricciones	Descripción
id	INT	Clave primaria, autoincremental	Identificador del vehículo
type	ENUM('car','yacht','helicopter')	Obligatorio	Categoría (coche, yate, helicóptero)
name	VARCHAR(100)	Obligatorio	Nombre del vehículo
description	TEXT		Descripción
price	DECIMAL(10,2)	Obligatorio	Precio por día
image_url	VARCHAR(255)		Imagen principal
is_available	BOOLEAN	Valor por defecto: TRUE	Disponible o no
created_at	TIMESTAMP	Valor por defecto: fecha actual	Fecha de alta

Tabla: item_images (imágenes de productos)

Guarda varias imágenes para cada vehículo, para el carrusel de la galería.

Campo	Tipo	Restricciones	Descripción
id	INT	Clave primaria, autoincremental	Identificador único
item_id	INT	Clave foránea → items(id), borrado en cascada	Vehículo al que pertenece
image_url	TEXT	Obligatorio	Ruta de la imagen
position	INT	Valor por defecto: 0	Orden en el carrusel

Tabla: orders (pedidos)

Registra los pedidos realizados por los usuarios.

Campo	Tipo	Restricciones	Descripción
id	INT	Clave primaria, autoincremental	Identificador del pedido
user_id	INT	Clave foránea → users(id), obligatorio	Usuario que hizo el pedido
total_price	DECIMAL(10,2)	Obligatorio	Total del pedido
status	VARCHAR(50)	Valor por defecto: 'completed'	Estado del pedido
created_at	TIMESTAMP	Valor por defecto: fecha actual	Fecha del pedido

Tabla: order_items (detalle del pedido)

Recoge los vehículos concretos incluidos en cada pedido (relación muchos a muchos entre pedidos y productos).

Campo	Tipo	Restricciones	Descripción
id	INT	Clave primaria, autoincremental	Identificador único
order_id	INT	Clave foránea → orders(id), obligatorio	Pedido al que pertenece
item_id	INT	Clave foránea → items(id), obligatorio	Vehículo reservado
quantity	INT	Valor por defecto: 1	Cantidad
price	DECIMAL(10,2)	Obligatorio	Precio en el momento de la reserva
start_date	DATE		Fecha de inicio del alquiler
end_date	DATE		Fecha de fin del alquiler

Tabla: messages (mensajes de contacto)

Almacena los mensajes enviados desde el formulario de contacto de la web.

Campo	Tipo	Restricciones	Descripción
id	INT	Clave primaria, autoincremental	Identificador único
name	VARCHAR(100)	Obligatorio	Nombre de quien escribe
email	VARCHAR(100)	Obligatorio	Correo electrónico
message	TEXT	Obligatorio	Contenido del mensaje
created_at	TIMESTAMP	Valor por defecto: fecha actual	Fecha de envío

Relaciones entre las tablas

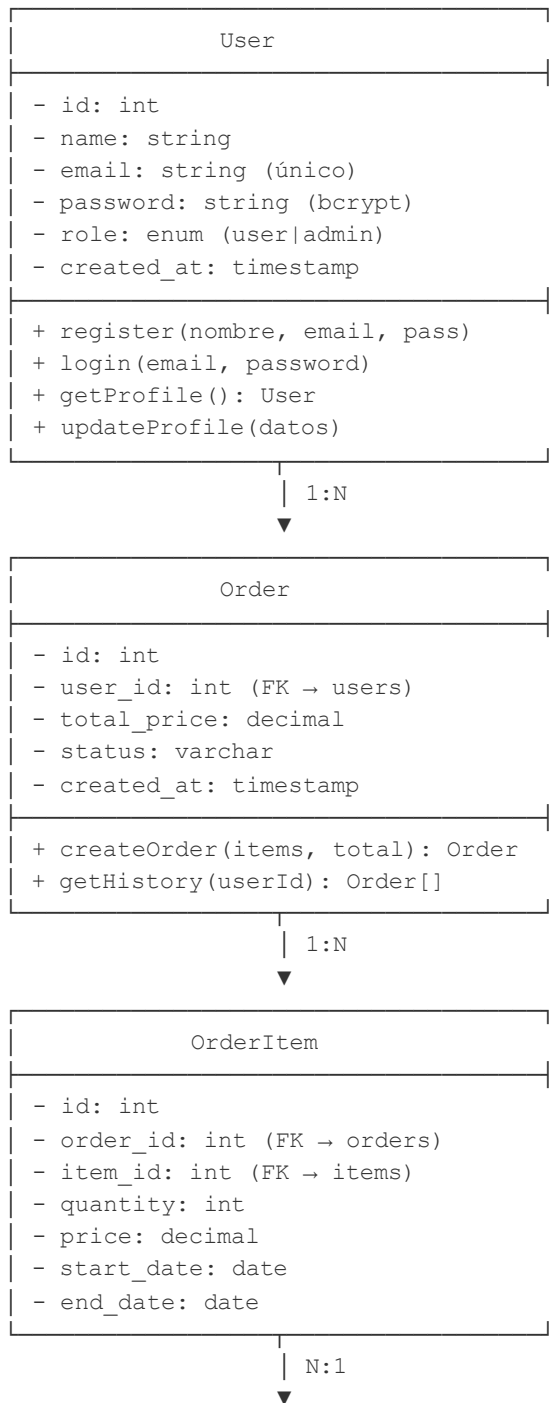
Las tablas se relacionan así:

- **users** → **orders**: un usuario puede hacer varios pedidos (1:N).
- **orders** → **order_items**: un pedido puede tener varias líneas (1:N).
- **items** → **order_items**: un vehículo puede estar en varios pedidos (1:N).
- **items** → **item_images**: un vehículo puede tener varias imágenes (1:N).

En la tabla item_images se ha definido el borrado en cascada, de forma que al eliminar un vehículo se borran automáticamente todas sus imágenes asociadas.

7.2. Diagrama de clases

Este es el diagrama de clases simplificado del sistema, con las entidades principales y sus relaciones:



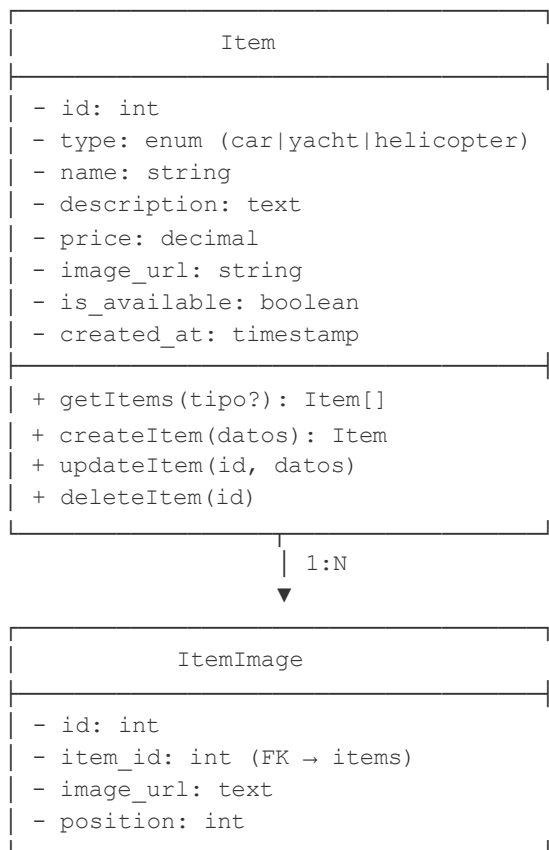


Tabla adicional independiente: messages (id, name, email, message, created_at)

Figura 1: Diagrama de clases del sistema RoyalRent.

7.3. Casos de uso

Estos son los casos de uso que recogen cómo los distintos actores interactúan con la aplicación:

ID	Caso de uso	Actor	Descripción
UC01	Registrarse	Visitante	Crear una cuenta con nombre, email y contraseña.
UC02	Iniciar sesión	Usuario	Acceder al sistema con email y contraseña.
UC03	Explorar catálogo	Usuario	Navegar por los vehículos filtrando por categoría.
UC04	Ver detalle	Usuario	Ver la información completa de un vehículo.
UC05	Añadir al carrito	Usuario	Añadir un vehículo al carrito de compra.
UC06	Gestionar carrito	Usuario	Cambiar cantidades o eliminar productos del carrito.
UC07	Reservar	Usuario	Completar el pago simulado y confirmar la reserva.
UC08	Editar perfil	Usuario	Modificar los datos personales.
UC09	Ver historial	Usuario	Consultar las reservas realizadas.
UC10	Gestionar usuarios	Administrador	Listar, crear y eliminar usuarios.
UC11	Gestionar productos	Administrador	Añadir, editar y eliminar vehículos.
UC12	Ver estadísticas	Administrador	Consultar métricas del negocio.
UC13	Gestionar mensajes	Administrador	Leer y eliminar mensajes de contacto.
UC14	Enviar mensaje	Visitante	Escribir una consulta mediante el formulario de contacto.

Tabla 1: Casos de uso del sistema.

7.4. Diseño de las interfaces

El diseño de RoyalRent busca ser elegante y funcional. La paleta de colores combina tonos oscuros con detalles en dorado como color de acento. Estas son las pantallas principales:

Página principal. Comienza con un hero que muestra un vídeo de fondo corporativo. Sobre él aparece el título de la web y un botón que lleva al catálogo. Más abajo hay estadísticas animadas, reseñas de clientes con efecto marquee infinito, tres tarjetas de servicios con efecto 3D, la presentación del equipo y un formulario de contacto funcional.

Catálogo de servicios. Los vehículos se muestran en una cuadrícula con pestañas para filtrar por categoría. Cada tarjeta incluye imagen, nombre, precio y disponibilidad. Al hacer clic se abre un panel lateral con la información completa del vehículo y un carrusel de imágenes.

Autenticación. Una sola página que combina inicio de sesión y registro, alternando entre ambos con una animación. Incluye validación de campos, mensajes de error claros y un indicador de carga.

Carrito y pago. El proceso va en dos pasos: primero se revisan los productos seleccionados y después se simula el pago. Se puede elegir entre varias tarjetas y se muestra un resumen con el total.

Perfil de usuario. Tiene dos columnas: a la izquierda un formulario para actualizar los datos personales, y a la derecha el historial de reservas, que se puede expandir para ver más detalles.

Panel de administración. Muestra estadísticas generales, gráficos por categorías, listado de pedidos recientes y tablas para gestionar usuarios, productos y mensajes.

7.5. Planificación del desarrollo

El proyecto se ha desarrollado en seis fases, siguiendo un enfoque progresivo:

Fase	Descripción	Duración
1. Análisis y diseño	Definir los requisitos, diseñar la base de datos y planificar la arquitectura.	2 semanas
2. Configuración del entorno	Preparar las herramientas, el servidor Express y la base de datos.	1 semana
3. Desarrollo del backend	Crear la API REST, la autenticación, el CRUD de usuarios y productos, y la lógica de pedidos.	3 semanas
4. Desarrollo del frontend	Construir los componentes React, las interfaces, las animaciones y la integración con la API.	4 semanas
5. Integración y pruebas	Probar el funcionamiento conjunto, corregir errores y optimizar.	2 semanas
6. Documentación	Escribir la memoria y preparar la defensa.	2 semanas

Tabla 2: Planificación del desarrollo.

7.6. Funcionalidad del proyecto

Sistema de autenticación

RoyalRent usa JSON Web Tokens (JWT) para la autenticación. Cuando un usuario se registra, el servidor valida los datos, cifra la contraseña con bcrypt (10 rondas) y la guarda en la base de datos. Al iniciar sesión, el servidor comprueba la contraseña y, si es correcta, genera un token con validez de 24 horas. Ese token se almacena en el navegador (localStorage) y se envía en cada petición mediante la cabecera Authorization. Un middleware se encarga de verificar el token en las rutas protegidas; si falta o es inválido, devuelve un error 401 o 403.

Catálogo

El catálogo muestra todos los vehículos y permite filtrarlos por tipo: coches, yates o helicópteros. Cada vehículo tiene varias imágenes en un carrusel, una descripción, especificaciones y el precio por día. Los administradores pueden añadir, editar y eliminar vehículos, y también cambiar su disponibilidad.

Reservas

El proceso de reserva tiene dos pasos: primero el usuario añade vehículos al carrito (que se guarda en localStorage para no perderse al cerrar el navegador), y después completa el pago simulado. Al confirmar, el servidor crea el pedido dentro de una transacción SQL: si algo falla, se deshace todo y los datos quedan intactos.

Panel de administración

El panel ofrece estadísticas (usuarios, pedidos, ingresos), gráficos por categorías, y tablas para gestionar usuarios, productos, pedidos y mensajes de contacto.

8. Conclusiones y Mejoras

8.1. Conclusiones

Una vez terminado el proyecto, se pueden destacar estos puntos:

- Se ha conseguido una aplicación web completa y funcional que cumple los requisitos planteados en el anteproyecto.
- La separación entre frontend y backend facilita el mantenimiento y permite reutilizar el backend con otras aplicaciones.
- La autenticación con JWT ofrece un mecanismo de seguridad sólido y adecuado para una API REST.
- El diseño responsivo garantiza una buena experiencia tanto en móvil como en escritorio.
- Las transacciones SQL aseguran que los pedidos se creen de forma íntegra, sin riesgo de datos inconsistentes.
- Se han aplicado conocimientos de los ocho módulos del ciclo, lo que ha permitido consolidar el aprendizaje en un proyecto real.

8.2. Mejoras propuestas

De cara al futuro, se podrían incorporar las siguientes mejoras:

- **Pago real:** integrar Stripe o PayPal para procesar pagos reales.
- **Notificaciones:** enviar correos de confirmación de reserva (el sistema nodemailer ya está configurado).
- **Calendario:** añadir un calendario interactivo para elegir fechas y ver disponibilidad en tiempo real.
- **Búsqueda avanzada:** filtrar por precio, marca, modelo o características.
- **Valoraciones:** permitir a los usuarios puntuar los vehículos que han alquilado.
- **Tests automáticos:** incorporar pruebas unitarias y de integración.

9. Bibliografía

- [1] Meta Platforms, Inc. *React Documentation*. 2025. <https://react.dev>
- [2] OpenJS Foundation. *Express.js Documentation*. 2025. <https://expressjs.com>
- [3] Oracle Corporation. *MySQL 8.0 Reference Manual*. 2025. <https://dev.mysql.com/doc>
- [4] OpenJS Foundation. *Node.js Documentation*. 2025. <https://nodejs.org>
- [5] Vite Contributors. *Vite Documentation*. 2025. <https://vitejs.dev>
- [6] Auth0. *Introduction to JSON Web Tokens*. 2025. <https://jwt.io>
- [7] NPM, Inc. *bcrypt*. 2025. <https://www.npmjs.com/package/bcrypt>
- [8] Remix Software. *React Router Documentation*. 2025. <https://reactrouter.com>
- [9] Axios Contributors. *Axios Documentation*. 2025. <https://axios-http.com>
- [10] NPM, Inc. *mysql2*. 2025. <https://www.npmjs.com/package/mysql2>
- [11] Framer Motion Contributors. *Framer Motion*. 2025. <https://www.framer.com/motion>

10. Anexos

Anexo A: Guía de instalación

Prerrequisitos

- Node.js (v18 o superior).
- MySQL 8.0 (XAMPP o similar).
- npm (incluido con Node.js).

Pasos

1. Arrancar MySQL (XAMPP → Start MySQL).
2. Crear el archivo `.env` en `backend/`:

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=royalrent
JWT_SECRET=royalrent_secret_key
PORT=5000
```

3. Inicializar la base de datos:

```
cd backend
npm install
npm run init-db
```

4. Iniciar el backend:

```
cd backend
npm start
```

5. Iniciar el frontend:

```
cd frontend
npm install
npm run dev
```

La web se abre en `http://localhost:5173` y la API en `http://localhost:5000`.

Anexo B: Datos de prueba

Al ejecutar `npm run init-db` se crean automáticamente estos datos:

- **Administrador:** admin@royalrent.com / admin123
- **40 vehículos:** 20 coches (Ferrari, Lamborghini, Rolls-Royce, Bentley, Porsche, Mercedes, Aston Martin, McLaren), 12 yates (Azimut, Sunseeker, Ferretti, Riva, Princess, Benetti) y 8 helicópteros (Bell, Airbus, AgustaWestland, Sikorsky).

Anexo C: Estructura del proyecto

```
royalrent/
├── backend/
│   ├── .env                # Variables de entorno
│   ├── package.json        # Dependencias
│   ├── server.js           # Servidor Express
│   ├── schema.sql          # Creación de la BD
│   ├── init_db.js          # Inicialización
│   ├── alter_db.js         # Migraciones
│   └── seed_item_images.js  # Imágenes
├── frontend/
│   ├── index.html          # Página principal
│   ├── package.json        # Dependencias
│   ├── vite.config.js      # Configuración Vite
│   ├── public/img/         # Imágenes
│   └── src/
│       ├── main.jsx        # Punto de entrada
│       ├── App.jsx         # Componente raíz
│       ├── index.css       # Estilos globales
│       ├── servicios.css   # Estilos catálogo
│       ├── components/     # Componentes
│       │   ├── Navbar.jsx
│       │   ├── Footer.jsx
│       │   └── ServiceDrawer.jsx
│       ├── pages/          # Páginas
│       │   ├── Home.jsx
│       │   ├── Services.jsx
│       │   ├── Auth.jsx
│       │   ├── Cart.jsx
│       │   ├── Profile.jsx
│       │   └── AdminDashboard.jsx
│       └── context/        # Contextos
│           ├── AuthContext.jsx
│           ├── CartContext.jsx
│           ├── useAuth.js
│           └── useCart.js
```

Figura 2: Estructura de directorios.

Anexo D: API endpoints

Método	Ruta	Autenticación	Descripción
POST	/api/register	No	Registrar usuario
POST	/api/login	No	Iniciar sesión
GET	/api/items	No	Listar vehículos (? type=car yacht helicopter)
POST	/api/contact	No	Enviar mensaje de contacto
GET	/api/user/profile	Sí	Ver perfil
PUT	/api/user/profile	Sí	Actualizar perfil
POST	/api/orders	Sí	Crear pedido
GET	/api/orders/history	Sí	Historial de pedidos
GET	/api/admin/stats	Admin	Estadísticas
GET	/api/admin/users	Admin	Listar usuarios
DELETE	/api/admin/users/:id	Admin	Eliminar usuario
GET	/api/admin/orders	Admin	Listar pedidos
GET	/api/admin/items	Admin	Listar vehículos
POST	/api/admin/items	Admin	Crear vehículo
PUT	/api/admin/items/:id	Admin	Actualizar vehículo
DELETE	/api/admin/items/:id	Admin	Eliminar vehículo
PATCH	/api/admin/items/:id/availability	Admin	Cambiar disponibilidad
GET	/api/admin/messages	Admin	Ver mensajes
DELETE	/api/admin/messages/:id	Admin	Borrar mensaje

Tabla 3: API REST completa.